

NO SQL

→ surge como respuesta a nuevas necesidades tecnológicas que comenzaron en relación al crecimiento de internet y apps globales.

empresas que manejaban grandes volúmenes de datos como motores de búsqueda y plataformas de comercio electrónico.

las 5 V del Big Data

volumen
velocidad
valor

variedad
veracidad

NoSQL no significa la eliminación total del modelo relacional del SQL.

El término se interpreta mejor como "Not only SQL".

Características No SQL

Base (NoSQL)

- basically available: Prioriza la disponibilidad del sistema.
- soft state: Permite que el estado de datos cambie de forma gradual
- eventual consistency: Busca alcanzar la consistencia eventual.

✓ alta disponibilidad

✓ escalabilidad

X consistencia inmediata no garantizada.

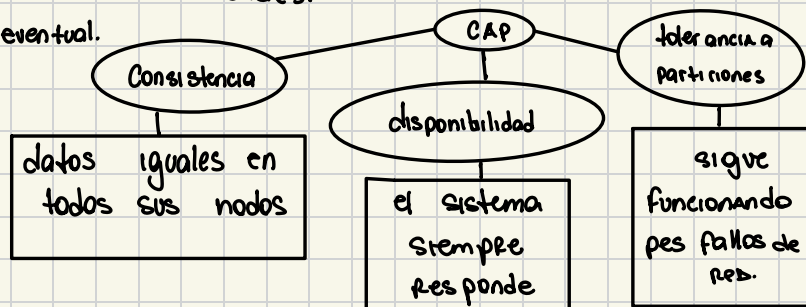
Usar NoSQL cuando

- Hay datos semi estructurados
- escala de datos masiva
- sistema requiere alta disponibilidad
- evolución rápida del modelo

➤ No cuando: Se requieren transacciones críticas
Integridad estricta (fuerte) de datos obligatoria.
Modelos altamente relacionales, complejos pero eficientes.

Teorema CAP

→ Solo se pueden garantizar simultáneamente dos de estas tres propiedades.



tipos de bases de datos No SQL

Clave - valor
columnar
→ vectoriales (enfocado en inteligencia artificial)

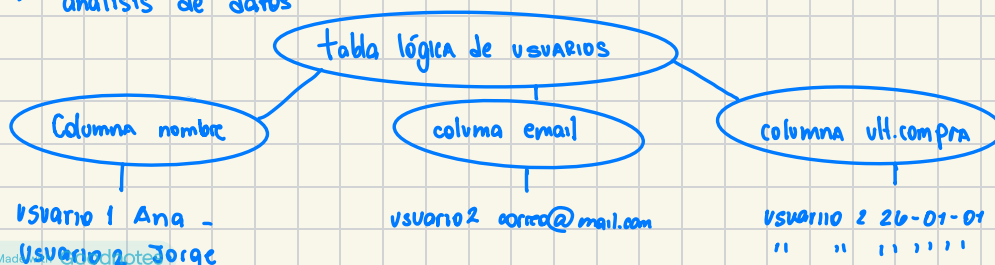
documentales
grafos

Bases de datos columnares

→ los datos se organizan en familias de columnas en lugar de filas rígidas.

Esto significa que cada registro puede contener diferentes columnas según sea necesario, lo que proporciona mayor flexibilidad que el modelo relacional tradicional.

- Buen rendimiento en grandes escalas
- optimiza almacenamiento
- mejora el rendimiento en operaciones de lectura y análisis
- análisis de datos



Clave-valor

Es el más simple dentro del mundo NoSQL
→ los datos se almacenan de forma similar a un diccionario como pares donde una clave única se asocia a un valor.

- no permite consultas complejas ya que no es posible filtrar por atributos internos del valor.
- no maneja relaciones entre datos

Suelen utilizarse en escenarios donde la rapidez es prioritaria y los datos no requieren análisis complejo

casos comunes: • manejo de sesiones de usuario
sistemas de caché

• almacenamiento temporal

• apps que necesitan respuestas en tiempo real.

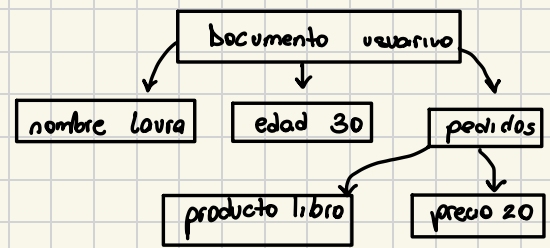
clave 1 → valor datos usuario
clave 2 → " " " → la base no
clave 3 → " " " → interpreta el contenido

Documentales

- Almacena información en documentos con formato similar a JSON.
Cada doc puede tener su propia estructura

→ Permiten representar datos complejos de forma flexible y cercana al modelo utilizado en aplicaciones modernas.

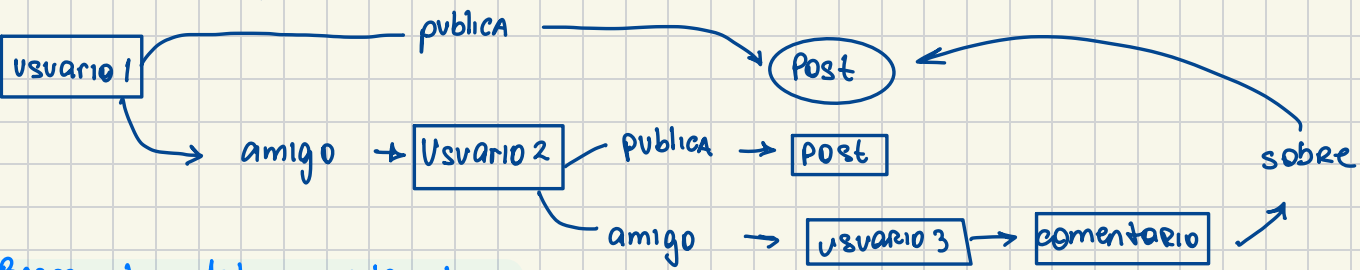
- flexible
 - evolución
 - buen soporte a consultas
- > Posible duplicación de info
> Relaciones complejas pueden ser costosas.



Bases de datos de grafos

Están diseñadas para manejar relaciones complejas entre entidades. Utilizan nodos y conexiones para representar los datos.

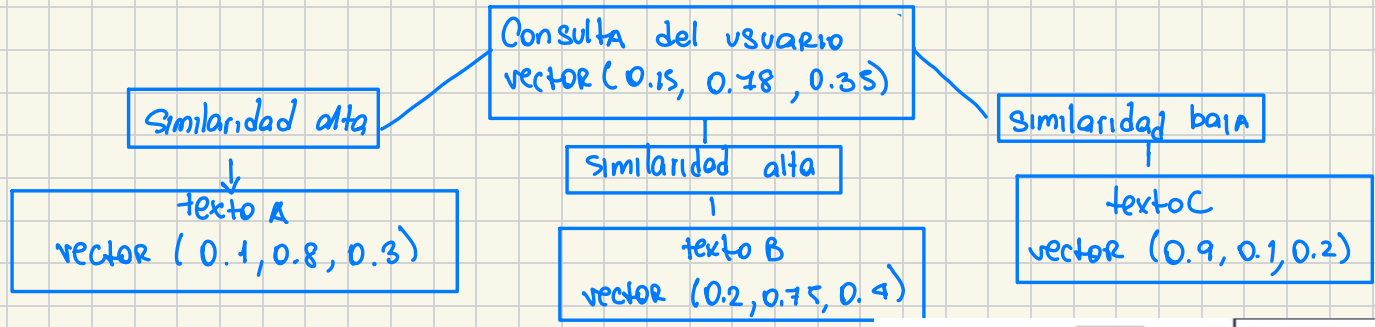
- permite explorar relaciones profundas de manera eficiente.
- buen manejo de relaciones
 - consultas complejas eficientes
- x no ideal para datos tabulares simples
x requieren cambios en el modelado



Bases de datos vectoriales

→ Almacenan información en forma de vectores numéricos que permiten medir similitud entre datos.
Se utilizan en inteligencia artificial y búsqueda semántica.

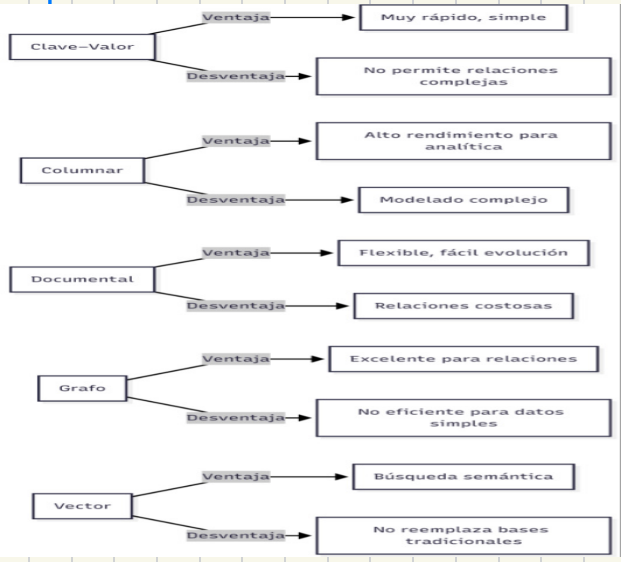
- permiten búsqueda por similitud
 - integración con sistemas de IA
- x requieren procesamiento previo de datos.



Casos de uso y elección de No SQL

- tipos de datos
- estructurados → SQL es suficiente
 - semiestructurados → documental o columnar
 - relaciones complejas → grafo
 - datos vectoriales → IA/ búsqueda semántica
- escala masiva de lecturas → columnar o clave-valor
- escrituras rápidas y concurrentes → clave-valor o documental

relaciones profundas → grafos simples → documental, SQL



Bases de datos clave-valor

→ se necesita almacenar y recuperar sesiones de usuario para una aplicación web con millones de accesos diarios.

cada sesión debe recuperarse en menos de 5ms, no requiere relaciones con otros datos y puede expirar automáticamente tras un periodo de inactividad.

→ el enunciado menciona

- accesos muy frecuentes
- latencia muy baja
- datos simples asociados a una clave
- sin relaciones entre entidades
- los datos caducan

Acceso directo y rápido a un valor conocido

UNA base de datos Clave-valor

porque: · permite acceso $O(1)$ por clave

- rápida
- soporta TTL (time to live)
- escala horizontalmente sin complejidad

Base de datos columnar

Una empresa recopila miles de millones de eventos diarios provenientes de sensores y aplicaciones.

· Se necesita realizar consultas analíticas agregadas por fecha, región y tipo de evento, priorizando velocidad de lectura sobre actualizaciones individuales.

→ patrones globales en grandes conjuntos de datos.

enunciado apunta a:

- volumen masivo de datos
- datos homogéneos y repetitivos
- consultas por rangos y agregaciones
- predominio de lecturas analíticas
- escritura por lotes, no transaccionales

Base de datos columnar

- optimiza lecturas por columnas específicas.
- Reduce I/O en consultas
- escala horizontalmente grandes volúmenes.

Agregaciones: Resumir grandes volúmenes de datos aplicando funciones matemáticas o estadísticas sobre una o varias columnas, sin trabajar registro por registro de forma individual.

Bases de datos documentales

→ Se debe construir un sistema catálogo para un e-commerce donde cada producto puede tener atributos distintos según su categoría.

· el modelo de datos puede cambiar con frecuencia y se requiere consultar productos por múltiples campos.

el enunciado destaca:

- entidades con estructura variable
- cambios frecuentes en el modelo
- consultas flexibles por distintos atributos
- datos naturalmente agrupados por entidad

Base de datos documental porque:

- permite esquemas dinámicos
- modela cada entidad como un documento autocontenido
- facilita evolución sin migraciones costosas
- soporta índices y consultas RICAS.

el punto clave es flexibilidad estructural y autonomía de cada entidad.

Bases de datos de grafos

- Se necesita analizar relaciones entre usuarios, transacciones y dispositivos para detectar patrones de fraude.
- es importante identificar caminos indirectos y relaciones del segundo o tercer nivel en tiempo casi real.

en grafos

- optimiza recorridos
- simplifica consultas
- análisis relacional profundo

términos clave

- relaciones complejas
- conexiones indirectas
- caminos entre entidades
- análisis relacional profundo

- un nodo puede existir sin relaciones
- una relación también puede existir con identidad propia
- ambas son partes fundamentales del modelo.

Bases de datos vectoriales

- Se requiere implementar un sistema de búsqueda semántica que permita encontrar documentos similares a una consulta de texto, incluso cuando no coinciden las palabras exactas.

→ El sistema se usará como base para un chatbot y un motor de recomendaciones.

El enunciado enfatiza

- similitud semántica
- comparación aproximada
- representación semántica del significado
- integración con IA

Base de datos vectorial necesaria para:

- almacena embeddings numéricos
- permite búsquedas por similitud
- está optimizado por alta dimensionalidad

Teoría de bases de datos documentales y fundamentos de MongoDB

Una base de datos documental almacena datos en documentos, generalmente en formato JSON o BSON.

El modelo documental parte de una idea distinta al modelo relacional.

→ en lugar de dividir la información en diferentes tablas relacionadas, agrupa los datos que pertenecen naturalmente a una misma entidad dentro de un único documento

MongoDB como implementación del modelo documental

BSON (representación binaria de JSON que permite tipos de datos adicionales

- mayor eficiencia de almacenamiento
- soporte a fechas y datos binarios).

Colección: conjunto de elementos similares

documento: JSON/BSON

campo: par clave valor dentro del documento

índice: estructura para acelerar consultas

- la unidad fundamental no es la fila ni la tabla, es el documento.

Indexación

- Permite indexar campos

el diseño en bases documentales se basa en:

- embedding: cuando se relacionan los datos
- referencias: cuando el volumen o reutilización lo requiere.

uso mongoDB cuando

- catálogos con atributos variable
- APIs que devuelven JSON

PAYLOAD

- cuerpo de datos que se envía dentro de una petición HTTP.

cliente → API → payload JSON → MongoDB → Documento

Relación entre JSON y BSON

- MongoDB almacena documentos en formato BSON

Respaldo o backup: copia controlada del estado de la base de datos en un momento determinado.

comando típico de respaldo

```
mongodump --uri="mongodb://admin:admin123@localhost:27016" --out=backup_universidad
```

```
mongodump --out=backup_universidad
```

- Se conecta al servidor MongoDB
- recorre las bases de datos accesibles
- guarda los archivos en un directorio local
- exporta cada colección a archivos BSON

Restauración de datos

```
mongorestore --drop
```

```
--uri="mongodb://admin:admin123@localhost:27016" backup_universidad
```

- lee los archivos BSON
- recrea las colecciones
- inserta nuevamente los documentos

Observar el comportamiento del sistema

```
db.stats()
```

```
db.serverStatus()
```

```
db.currentOp()
```

```
db.collection.stats()
```

Auditoría: registra qué usuarios operan en el sistema.

Creación de usuarios en MongoDB

- nombre
- contraseña
- rol asignado

```
use admin
```

```
db.createUser({
```

```
  user: "admin66"
```

```
  pwd: "123@"
```

```
  roles: [
```

```
    {role: "userAnyDatabase", db: "admin"}]
```

```
}]
```

Roles predefinidos en MongoDB

- Read (no cambios)
- ReadWrite (sí cambios)
- DbAdmin (administra índices y operaciones administrativas)
- ClusterAdmin (administra todo el cluster)

Principio del mínimo privilegio

- un usuario o sistema debe tener únicamente los permisos necesarios para realizar su función.

validación de esquema con JSON Schema

```
db.createCollection("cursos", {
```

```
  validator: {
```

```
    $jsonSchema: {
```

```
      bsonType: "object",
```

```
      required: ["nombre", "nivel", "creditos"],
```

```
      properties: {
```

```
        nombre: {
```

```
          bsonType: "string",
```

```
          description: "Debe ser un texto"
```

```
        },
```

```
        nivel: {
```

```
          enum: ["basico", "intermedio", "avanzado"]
```

```
        },
```

```
        créditos: {
```

```
          bsonType: "int",
```

```
          minimum: 1,
```

```
          maximum: 10
```

```
        }
```

```
      }
```

```
    }
```

```
  }
```

```
})
```

strict y moderate

strict: rechaza lo que no cumple el esquema

moderate: solo valida nuevos o modificados

ejemplo de migración simple

```
db.cursos.updateMany(  
  { créditos: { $exists: false } },  
  { $set: { créditos: 3 } }
```

NOTA: los índices no son gratuitos

- consumen espacio
- tiempo

Creación de índices

· simples

```
db.cursos.createIndex({ nombre: 1 })
```

ver índices creados

```
db.cursos.getIndexes()
```

índices compuestos

```
db.cursos.createIndex({ nivel: 1, credits: -1 })
```

explain : analiza ejecución

```
db.cursos.find({ nivel: "Avanzado" }).explain('executionStats')
```

collScan: MongoDB recorrió toda la colección

IXScan: uso de índice

totalDocsExamined: docs que evaluó

totalKeysExamined: no. entradas del índice evaluadas

Índices únicos

```
db.usuarios.createIndex({ email: 1, { unique: true } })
```

\$unwind - desnormalizando arrays

```
db.cursos.aggregate([
  { $unwind: "$tecnologias" },
  {
    $group: {
      _id: "$tecnologias",
      total: { $sum: 1 }
    }
  },
  { $sort: { total: -1 } }
])
```

Si el array es grande \$unwind puede multiplicar el volumen de datos en el pipeline.

ORDEN PIPELINE

\$match
\$project
\$group
\$sort
\$limit

→ cuando agrupamos grandes volúmenes:

- \$group: mantener estructura de memoria
- \$sort: requiere buffers grandes

consulta

```
db.cursos.find().sort({ credits: 1 })
```

con índice

```
db.cursos.createIndex({ credits: 1 })
```

Índices en campos embebidos

crear

```
db.estudiantes.createIndex({ "direccion.ciudad": 1 })
```

consulta

```
db.estudiantes.find({ "direccion.ciudad": "madrid" })
```

\$lookup - join en MongoDB

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "clienteId",
      foreignField: "_id",
      as: "clienteInfo"
    }
  }
])
```